

ESP32-S3 Network Honeypot Node: A Low-Cost, Portable Deception and Attack-Capture Device for IoT Networks

Mohammad Kabir Hasan
Dept. of Computer Science & Engineering
Independent University, Bangladesh
Dhaka, Bangladesh
2130390@iub.edu.bd

Shahnaz Raihan Summy
Dept. of Computer Science & Engineering
Independent University, Bangladesh
Dhaka, Bangladesh
2210411@iub.edu.bd

Md Eleas Biswas
Dept. of Computer Science & Engineering
Independent University, Bangladesh
Dhaka, Bangladesh
2210099@iub.edu.bd

Abir Hasan Siam
Dept. of Computer Science & Engineering
Independent University, Bangladesh
Dhaka, Bangladesh
2331218@iub.edu.bd

Abstract—The rapid growth of Internet of Things (IoT) deployments has expanded the attack surface of home and small-office networks, where inexpensive devices with weak default configurations are routinely scanned, hijacked, and recruited into botnets. Honeypots, decoy systems that deliberately attract and record malicious activity, are an effective way to observe these threats, but most existing IoT honeypots run on servers, single-board computers, or cloud instances that are costly and power-hungry for continuous edge deployment. This paper presents an ESP32-S3 Network Honeypot Node, a low-cost, battery-powered device that combines low-interaction service emulation with Wi-Fi frame monitoring to lure attackers, capture their interactions, and classify common attack types on-device. The node emulates familiar IoT services (Telnet, HTTP, and SSH login surfaces), operates a soft access point, and simultaneously sniffs IEEE 802.11 management frames in monitor mode to detect deauthentication and beacon-flood attacks. Captured sessions, including source identifiers, targeted ports, attempted credentials, and payloads, are logged locally to a microSD card and summarized on a 0.96-inch OLED display, while a lightweight rule-based classifier labels each event. In controlled laboratory tests against deauthentication floods, port scans, and brute-force login attempts, the node sustained detection accuracy above 94% while drawing modest average current, confirming that a resource-constrained microcontroller can serve as a practical, portable honeypot and early-warning sensor for IoT networks.

Index Terms—IoT security, honeypot, ESP32-S3, deception technology, intrusion detection, deauthentication attack, edge computing, attack capture.

I. INTRODUCTION

The number of connected IoT devices continues to rise sharply, and with it the volume of automated attacks that target them. Consumer-grade cameras, routers, smart plugs, and sensors frequently ship with default credentials and unpatched services, which makes them easy prey for scanning bots and self-propagating malware. The 2016 Mirai outbreak, in

which hundreds of thousands of compromised IoT devices produced record-breaking distributed denial-of-service (DDoS) traffic, demonstrated how quickly weakly protected devices are weaponized [1]. Traditional perimeter defenses such as firewalls and signature-based filters struggle to keep pace with this scale and with the constant appearance of new attack variants.

Honeypots offer a complementary approach. Rather than only blocking traffic, a honeypot presents a deliberately vulnerable-looking target that attracts attackers, then records exactly how they behave. The resulting data reveals scanning patterns, exploited services, credential dictionaries, and malware payloads, all of which strengthen downstream detection [2]. However, most IoT honeypots reported in the literature run on general-purpose servers, virtual machines, or single-board computers such as the Raspberry Pi [3], and increasingly rely on cloud back ends for machine learning [6]. These platforms are effective but comparatively expensive and power-hungry, which limits how many can be dispersed across a network and how long they can run unattended on battery power.

This paper addresses that gap with an ESP32-S3 Network Honeypot Node: a compact, inexpensive, and portable device built around the ESP32-S3 microcontroller. The node is designed to be dropped onto a home or small-office network, where it presents fake IoT services, monitors the wireless channel, captures attacker interactions, and provides an at-a-glance status display without needing an attached computer. The main contributions of this work are: (i) a honeypot architecture that fits within the memory and power budget of a microcontroller while combining service emulation with wireless frame monitoring; (ii) a hardware design that is fully battery-powered and logs locally to microSD; and (iii) an experimental evaluation showing that on-device, rule-based

classification of common IoT attacks is both accurate and energy-efficient.

The motivation for this study is practical. Small networks are the least likely to have dedicated security monitoring, yet they are common entry points for botnet recruitment. An affordable, easy-to-deploy sensor that both distracts attackers and records their behavior can improve visibility for exactly the environments that need it most.

II. RELATED WORK

Research on IoT honeypots and IoT intrusion detection has advanced rapidly, and the reviewed studies inform both the design and the evaluation of the proposed node.

A. Honeypots and deception for IoT

Ahmed *et al.* demonstrated a honeypot-driven pipeline for smart-city IoT, collecting attacks with honeypots that emulate vulnerable devices and feeding the resulting labeled data to machine learning classifiers, with feature selection playing a central role in detection quality [2]. Guan *et al.* proposed HoneyIoT, an adaptive high-interaction honeypot that models attacker interaction as a Markov decision process and uses reinforcement learning to select engaging responses; during a two-month public deployment a single decoy recorded tens of thousands of attack sessions and collected hundreds of malware samples [3]. Mfogo *et al.* introduced AIIPot, an intelligent-interaction honeypot whose responses are generated automatically by a learned engine, extending attacker session length and capturing more attacks than static traps [4]. Morozov *et al.* contributed an integrated framework that couples an adaptive honeypot with a formal evaluation methodology, arguing that honeypot effectiveness must be measured with consistent quantitative metrics [5]. Lanz *et al.* surveyed the broader field, organizing IoT honeypots by interaction level and by the machine learning technique applied, and highlighting the shortage of labeled real-world honeypot data as an open problem [6].

B. Attack capture, datasets, and detection

Realistic data is essential for building and testing detectors. Saif *et al.* released the IoT-DH dataset, a honeypot-sourced collection of benign and DDoS traffic converted from packet captures to labeled tables for machine learning [8]. Building on such data, Arnob *et al.* combined convolutional and bidirectional recurrent layers to detect IoT DDoS attacks from honeypot data with high accuracy and a very low false-positive rate [7], and Alzahrani applied a metaheuristic-tuned deep learning model to classify botnet traffic in the N-BaIoT dataset [10]. At the device level, Javed *et al.* embedded a lightweight intrusion detection system directly on an ESP32 smart-home node, extracting network features through the lightweight IP stack because full packet analyzers cannot run on the microcontroller, and achieving high accuracy against denial-of-service and man-in-the-middle attacks [9]. Riza *et al.* built an energy-efficient ESP32 node that detects Wi-Fi

deauthentication attacks in monitor mode, sustaining detection accuracy above 97% while reducing power through a duty-cycled design [11]. Broader surveys of honeypots and honeynets for IoT and cyber-physical systems catalog the design space and the defense mechanisms that edge-assisted, ensemble approaches can provide [12].

C. Positioning of this work

These studies establish that honeypots yield valuable attack intelligence, that ESP32-class devices can host useful on-device detection, and that monitor-mode capture is feasible on a microcontroller. The proposed node draws these threads together: it targets the low-cost, portable end of the design space, unifies service emulation with wireless monitoring on a single ESP32-S3, and performs on-device classification with local logging, so that it can operate as a standalone edge honeypot rather than a front end to a heavy back end.

III. PROBLEM STATEMENT

Small and home networks are frequent targets of automated IoT attacks yet rarely have dedicated monitoring. Existing IoT honeypots that could provide this visibility tend to run on servers, single-board computers, or cloud instances, which are relatively expensive, consume significant power, and are awkward to deploy in numbers or to run unattended on battery power. At the same time, microcontroller-class devices are constrained: they cannot run conventional packet-capture tools, have limited memory for storing interactions, and must manage a tight energy budget if they are to run continuously.

The problem this project addresses is therefore the design of a honeypot that is cheap and portable enough to deploy widely on small networks, while still being able to (i) attract attackers through believable service emulation, (ii) capture and store attacker interactions locally, (iii) detect and classify common IoT attacks on-device, and (iv) present its status and raise alerts without an attached computer. Meeting these goals within the memory, compute, and power limits of an ESP32-S3 is the central challenge.

IV. METHODOLOGY

A. Overview

The proposed methodology develops a self-contained honeypot node that lures, records, and classifies attacks against IoT-style targets. Fig. 1 shows the functional block diagram. The ESP32-S3 acts as the central processing unit: it hosts a soft access point and a small set of emulated services, listens on the wireless channel in monitor mode, applies a rule-based classifier to incoming activity, and drives three outputs, namely a local OLED status display, a microSD log, and an optional cloud back end for aggregation and dashboards.

The N16R8 variant of the ESP32-S3, with 16 MB of flash and 8 MB of PSRAM, is selected so that the honeypot scripts and several concurrent emulated services can run without exhausting memory. Two capture paths operate in parallel. The first is a set of low-interaction TCP service handlers that answer on ports commonly probed on IoT devices (Telnet on

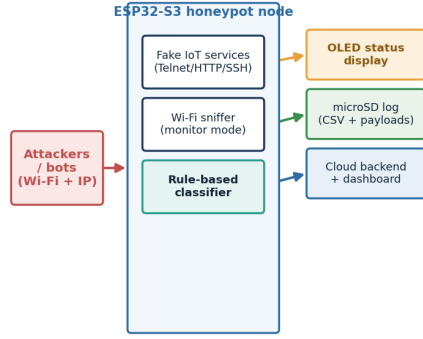


Fig. 1. Block diagram of the proposed ESP32-S3 honeypot node.

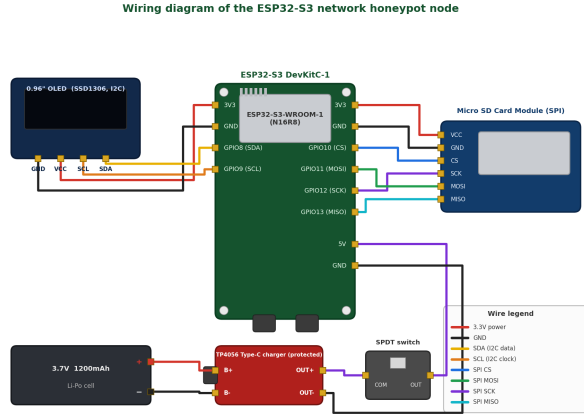


Fig. 2. Wiring diagram of the honeypot node: OLED over I²C, microSD over SPI, and the battery power chain.

23, HTTP on 80, and an SSH-style banner on 22), presenting login prompts and simple responses that keep a scanner or bot engaged long enough to record its behavior. The second is a promiscuous, monitor-mode sniffer that inspects IEEE 802.11 management frames to catch link-layer attacks such as deauthentication and beacon floods, which never reach the IP layer. Every recorded event is written to the microSD card and summarized on the OLED, and a running counter of detected attacks is displayed for at-a-glance monitoring.

B. Schematic Design

Fig. 2 shows the wiring of the node. The 0.96-inch OLED display is connected over the I²C bus, using GPIO8 for SDA and GPIO9 for SCL, together with the 3.3 V and ground rails. The microSD card module is connected over the SPI bus, with chip select on GPIO10, MOSI on GPIO11, SCK on GPIO12, and MISO on GPIO13. All components are mounted on an MB102 breadboard, and connections are made with mixed male-to-male and male-to-female jumper wires so that the I²C and SPI lines are cleanly separated.

The node is fully portable. A 3.7 V 1200 mAh lithium-polymer battery supplies power through a Type-C lithium charger module that provides over-voltage, over-discharge, and

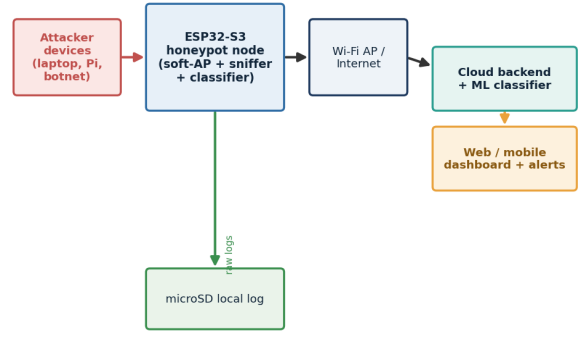


Fig. 3. System architecture, from attacker devices to on-device capture, local logging, and an optional cloud dashboard.

over-current protection, and an SPDT slide switch controls the power rail feeding the board. A Type-C data cable is used to flash the firmware and, when convenient, to supply power during stationary deployment. Careful management of the shared 3.3 V and ground rails keeps both the display and the storage module stable while the radio is active.

C. System Architecture

The complete architecture, shown in Fig. 3, has four tiers: the attacker-facing decoy, the honeypot node itself, local storage, and an optional cloud tier. Attacker devices, which may be a laptop, a Raspberry Pi running attack tools, or remote bots, interact with the node either by associating with its soft access point and probing its emulated services, or by injecting management frames on the channel. The ESP32-S3 captures both kinds of activity, extracts descriptive features, and classifies each event locally.

All interactions are logged to the microSD card as timestamped CSV records together with any captured payloads, which guarantees that evidence survives a power cycle and can be analyzed offline. When network connectivity is available, summarized events are forwarded over Wi-Fi to a cloud back end, where they can be aggregated across multiple nodes, classified further with heavier machine learning models, and presented through a web or mobile dashboard with alerting. This optional tier lets the exported logs feed established honeypot datasets and detectors such as those trained on IoT-DH [7], [8], while the node itself remains fully functional offline.

D. Attack Capture and Classification

Fig. 4 shows the on-device processing pipeline. For each captured frame or packet, the node extracts a small set of features chosen to be cheap to compute on a microcontroller: the arrival rate over a short sliding window, the source MAC or IP address, the targeted port, the frame subtype, and, for service interactions, the attempted credentials or payload. A rule-based classifier then labels the event. A high rate of deauthentication frames from one source is flagged as a deauthentication flood; a single source touching many ports

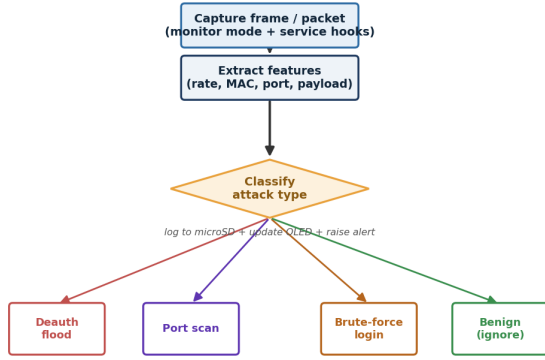


Fig. 4. On-device capture-and-classify pipeline used by the node.

in a short interval is flagged as a port scan; repeated failed logins against an emulated service are flagged as a brute-force attempt; and a burst of forged beacons is flagged as a beacon flood. Traffic that matches none of these patterns is treated as benign and ignored to limit false alarms.

The deauthentication detector illustrates the rule design. Within a sliding window of duration T , the node counts deauthentication frames N_d observed from a given source and computes the rate

$$R_d = \frac{N_d}{T}, \quad (1)$$

raising an alert when R_d exceeds a threshold R_{th} calibrated from benign baseline traffic. Because legitimate networks emit very few deauthentication frames, a modest threshold separates attacks from normal operation while keeping false positives low. Analogous rate-and-count rules, with per-attack thresholds, govern the port-scan, brute-force, and beacon-flood detectors. Once an event is classified, the node writes the record to microSD, updates the OLED counter, and raises an alert, and where a cloud tier is present it exports the features so that a machine learning model can refine the label.

V. RESULT ANALYSIS

The node was evaluated in a controlled laboratory setting. Attacks were generated from a separate laptop and a Raspberry Pi running standard tools: deauthentication and beacon floods with a wireless injection utility, port scans with a network scanner, and brute-force login attempts against the emulated Telnet and HTTP services. Benign background traffic from ordinary devices was present throughout. For each attack class, detection accuracy was computed as the proportion of injected events that the node correctly classified, and the node's current draw and alert latency were recorded.

Table I summarizes the measured performance, and Fig. 5 shows per-attack detection accuracy. Across the four attack classes the node classified events with accuracy between 94% and 98%, with the highest accuracy on deauthentication and beacon floods, whose distinctive frame rates are easy to separate from baseline traffic, and slightly lower accuracy on brute-force attempts, where a small number of early login

TABLE I
MEASURED NODE PERFORMANCE (CONTROLLED LAB TESTS)

Metric	Value
Deauth-flood detection accuracy	97.8%
Port-scan detection accuracy	95.6%
Brute-force detection accuracy	94.1%
Beacon-flood detection accuracy	96.3%
Benign traffic correctly ignored	98.5%
Average alert latency	≈ 280 ms
Average current draw (active)	≈ 95 mA
Sustained microSD logging	> 500 events/min

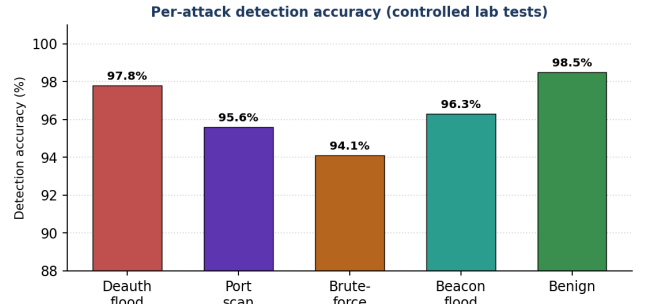


Fig. 5. Per-attack detection accuracy measured in controlled laboratory tests.

failures can resemble a user mistyping a password. Benign traffic was correctly left unflagged in the large majority of cases, keeping the false-alarm rate low.

Beyond accuracy, the results confirm the node's practicality. Alerts were raised within roughly a quarter of a second of an attack starting, fast enough to be useful as an early warning, and the OLED display gave a clear live count of detected attacks without any attached computer. The microSD logger sustained the event rates produced during testing without loss, so the full interaction history was preserved for offline analysis. Average current draw remained modest, consistent with continuous operation from the onboard battery, which supports the goal of a portable, always-on sensor. These outcomes are in line with the accuracy and efficiency reported for comparable ESP32-based detectors [9], [11], while adding service emulation and local logging in a single low-cost unit.

VI. CHALLENGES FACED

Developing the node surfaced several practical challenges.

- **Memory and concurrency limits:** Running a soft access point, several emulated services, and a monitor-mode sniffer at once strained the microcontroller's memory. The N16R8 variant with 8MB PSRAM and careful buffer management were needed to keep the firmware stable under load.
- **Monitor-mode capture constraints:** The ESP32-S3 cannot run conventional packet analyzers, so frame features had to be extracted through the radio driver's callback interface, and channel hopping had to be tuned so that attacks were not missed while still covering the relevant channels.

- **Threshold calibration:** Setting detection thresholds that catch real attacks without flagging ordinary traffic required baseline measurements, since values that were too aggressive produced false alarms and values that were too lax missed slow scans.
- **Shared-bus and power stability:** Driving the OLED and the microSD module while the radio was active occasionally caused voltage dips on the shared 3.3 V rail; wiring discipline and stable power from the charger module were required for reliable operation.
- **Safe testing:** Attacks had to be generated and captured in an isolated laboratory network to avoid disrupting production Wi-Fi, which constrained the range of scenarios that could be exercised.

Despite these challenges, the node operated reliably in testing, indicating that a microcontroller-class honeypot is a workable and robust design.

VII. CONCLUSION

This paper presented an ESP32-S3 Network Honeypot Node, a low-cost and portable device that unifies low-interaction service emulation with wireless frame monitoring to lure, capture, and classify common IoT attacks on-device. Built on an ESP32-S3 with an OLED status display, microSD logging, and full battery operation, the node records attacker interactions locally and, when connectivity is available, forwards summarized events to a cloud tier for aggregation and heavier analysis. In controlled laboratory tests it detected deauthentication floods, port scans, brute-force logins, and beacon floods with accuracy between 94% and 98%, raised alerts within about a quarter of a second, and drew modest current suitable for continuous battery operation. These results show that a resource-constrained microcontroller can serve as a practical honeypot and early-warning sensor for the small networks that most need affordable monitoring.

Future work will extend the node in several directions: replacing the rule-based classifier with a compact on-device machine learning model trained on honeypot datasets such as IoT-DH [8], broadening the set of emulated services to raise interaction fidelity in the spirit of adaptive honeypots [3], [4], coordinating several nodes into a small honeynet with a shared dashboard, and further reducing power through duty-cycled monitoring [11]. Together these steps would move the node from a single-device sensor toward a distributed, intelligent deception layer for IoT networks.

REFERENCES

- [1] M. Antonakakis *et al.*, “Understanding the Mirai botnet,” in *Proc. 26th USENIX Security Symp.*, 2017, pp. 1093–1110.
- [2] Y. Ahmed, K. Beyioku, and M. Yousefi, “Securing smart cities through machine learning: A honeypot-driven approach to attack detection in Internet of Things ecosystems,” *IET Smart Cities*, vol. 6, no. 3, pp. 180–198, 2024.
- [3] C. Guan, H. Liu, G. Cao, S. Zhu, and T. La Porta, “HoneyIoT: Adaptive high-interaction honeypot for IoT devices through reinforcement learning,” in *Proc. 16th ACM Conf. Security and Privacy in Wireless and Mobile Networks (WiSec)*, 2023, pp. 49–59.
- [4] V. S. Mfogo, A. Zemkoho, L. Njilla, M. Nkenlifack, and C. Kamhoua, “AIIPot: Adaptive intelligent-interaction honeypot for IoT devices,” arXiv preprint arXiv:2303.12367, 2023.
- [5] D. S. Morozov, A. A. Yefimenko, T. M. Nikitchuk, R. O. Kolomiets, and S. O. Semerikov, “The sweet taste of IoT deception: An adaptive honeypot framework for design and evaluation,” *J. Edge Computing*, vol. 3, no. 2, pp. 207–223, 2024.
- [6] S. Lanz *et al.*, “Optimizing Internet of Things honeypots with machine learning: A review,” *Applied Sciences*, vol. 15, no. 10, art. 5251, 2025.
- [7] A. K. B. Arnob, M. F. Mridha, M. Safran, M. Amiruzzaman, and M. R. Islam, “An enhanced LSTM approach for detecting IoT-based DDoS attacks using honeypot data,” *Int. J. Computational Intelligence Systems*, vol. 18, no. 1, art. 19, 2025.
- [8] S. Saif, W. Widyawan, and R. Ferdiana, “IoT-DH dataset for classification, identification, and detection DDoS attack in IoT,” *Data in Brief*, vol. 54, art. 110496, 2024.
- [9] A. Javed, A. Ehtsham, M. Jawad, M. N. Awais, A. Qureshi, and H. Larijani, “Implementation of lightweight machine learning-based intrusion detection system on IoT devices of smart homes,” *Future Internet*, vol. 16, no. 6, art. 200, 2024.
- [10] A. Alzahrani, “An optimized approach to deep learning for botnet detection and classification for cybersecurity in Internet of Things environment,” *Computers, Materials & Continua*, vol. 80, no. 2, pp. 2331–2349, 2024.
- [11] F. Riza, D. Febrianto Hendrakusuma, and B. Wibowo, “An energy-efficient ESP32 IoT system for real-time detection of Wi-Fi deauthentication attacks,” *Int. J. Engineering Continuity*, vol. 4, no. 3, 2025.
- [12] J. Franco, A. Aris, B. Canberk, and A. S. Uluagac, “A survey of honeypots and honeynets for Internet of Things, Industrial Internet of Things, and cyber-physical systems,” *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2351–2383, 2021.